Vizuelizacija prohodnosti u pravougaoniku sa preprekama

MATF Projekat

April 8, 2026

Sadržaj

1 Definisanje problema **2**

2 Algoritam **2**

2.1 Konstrukcija grafa krugova 2

2.2 Detekcija barijere (BFS pretraga) 3

3 Vizuelizacija putanje (Raylib) **3**

3.1 Diskretni BFS na mreži piksela 3

3.2 Rekonstrukcija i crtanje 4

4 Vizuelni prikaz i analiza test primera **4**

5 Mogućnosti primene **7**

1 Definisanje problema

Problem je preuzet sa platforme *LeetCode* (zadatak br. 3235: *Check if the Rectangle Corner Is Reachable*).

Dati su pravougaonik u 2D ravni sa donjim levim uglom u $(0, 0)$ i gornjim desnim uglom u (X_{corner}, Y_{corner}), kao i niz krugova $circles$ gde svaki krug $circles[i] = [x_i, y_i, r_i]$ predstavlja prepreku sa centrom (x_i, y_i) i poluprečnikom r_i.

Cilj je utvrditi da li postoji putanja unutar pravougaonika koja povezuje tačke $(0, 0)$ i (X_{corner}, Y_{corner}) pod sledećim uslovima:

- Putanja mora biti u potpunosti sadržana unutar granica pravougaonika: $0 \leq x \leq X_{corner}$ i $0 \leq y \leq Y_{corner}$.
- Putanja ne sme prolaziti kroz unutrašnjost niti dodirivati granicu bilo kog od datih krugova.
- Krugovi mogu imati centre unutar ili izvan pravougaonika, ali samo njihovi delovi koji se preklapaju sa pravougaonikom deluju kao prepreke.

Matematički, problem se svodi na proveru da li skup krugova formira neprekidnu barijeru koja razdvaja početnu tačku od ciljne. Barijera nastaje ako lanac krugova povezuje gornju ivicu $(y = Y_{corner})$ ili levu ivicu $(x = 0)$ sa, donjom ivicom $(y = 0)$ ili desnom ivicom $(x = X_{corner})$.

2 Algoritam

Rešenje problema prohodnosti pravougaonika zasniva se na detekciji lanca prepreka (krugova) koji povezuju ivice tako da zatvaraju prolaz. Algoritam se izvršava u dve ključne faze: konstrukcija grafa i pretraga putanje.

2.1 Konstrukcija grafa krugova

U ovoj fazi, svaki krug se posmatra kao čvor u grafu. Granu između dva čvora povlačimo ako se krugovi seku ili dodiruju, ali uz dodatni uslov da se ta tačka dodira nalazi unutar granica pravougaonika definisanog sa $xCorner$ i $yCorner$.

Glavni matematički uslov za presek dva kruga je da rastojanje između njihovih centara bude manje ili jednako zbiru njihovih poluprečnika:

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2} \leq r_1 + r_2$$

U kodu se, radi preciznosti i izbegavanja računanja korena, koristi kvadratna forma:

$$(x_1 - x_2)^2 + (y_1 - y_2)^2 \leq (r_1 + r_2)^2$$

```
for (int i = 0; i < n; i++) {
    for (int j = i + 1; j < n; j++) {

        if (intersectCircles(circles[i], circles[j])) {

            if (intersectInside(circles[i], circles[j], xCorner, yCorner)) {
                circle_graph[i].push_back(j);
                circle_graph[j].push_back(i);
            }
        }
    }
}
```

Listing 1: Konstrukcija grafa

2.2 Detekcija barijere (BFS pretraga)

Nakon što je graf formiran, problem prohodnosti se svodi na proveru da li postoji putanja u grafu koja spaja "početne" ivice sa "završnim" ivicama.

- **Izvor (Source):** Krugovi koji dodiruju levu ili gornju ivicu pravougaonika (blokiraju ulaz ili gornji deo).
- **Cilj (Target):** Krugovi koji dodiruju desnu ili donju ivicu (blokiraju izlaz ili donji deo).

Koristimo BFS algoritam (Breadth-First Search) kako bismo pokušali da pronađemo put od bilo kog "izvornog" kruga do bilo kog "ciljnog" kruga. Ako takva putanja postoji, prolaz je nemoguć.

```

while (!q.empty()) {
    int i = q.front();
    q.pop();

    if (target_nodes.count(i)) {
        return false;
    }

    for (int neighbor : circle_graph[i]) {
        if (!visited[neighbor]) {
            visited[neighbor] = true;
            q.push(neighbor);
        }
    }
}

return true;
```

Listing 2: BFS pretraga za proveru blokade

3 Vizuelizacija putanje (Raylib)

Za grafički prikaz rešenja koristi se **Raylib** biblioteka. Dok primarni algoritam samo utvrđuje *da li* putanja postoji (koristeći graf krugova), vizuelizacija zahteva pronalaženje i iscrtavanje konkretne putanje od početne tačke do krajnje tačke koja ne seče nijednu prepreku.

3.1 Diskretni BFS na mreži piksela

Da bismo pronašli vizuelnu putanju, implementiran je drugi BFS algoritam koji radi direktno nad koordinatama ekrana. Umesto krugova kao čvorova, ovaj algoritam tretira ekran kao mrežu tačaka (grid) sa korakom $step = 5$ piksela.

- **Stanje:** Svaki čvor je struktura `Point2D` koja sadrži celobrojne x i y koordinate na ekranu.
- **Susedstvo:** Iz svake tačke BFS pokušava kretanje u 8 smerova (horizontalno, vertikalno i dijagonalno).
- **Kolizija:** Tačka se smatra nevalidnom ako se nalazi unutar bilo kog kruga, uz dodatu malu marginu bezbednosti radi bolje vidljivosti linije.

```

int dx[] = {-step, step, 0, 0, -step, -step, step, step};
int dy[] = {0, 0, -step, step, -step, step, -step, step};

while (!q.empty()) {
    Point2D current = q.front();
    q.pop();

    if (current == end) { foundPath = true; break; }

```

```
    for (int i = 0; i < 8; ++i) {
        Point2D next = {current.x + dx[i], current.y + dy[i]};

        bool inCircle = false;
        for (const auto& c : circles) {
            float dist2 = pow(probX - c[0], 2) + pow(probY - c[1], 2);
            float proveraR = (float)c[2] + margina;

            if (dist2 <= proveraR * proveraR) {
                inCircle = true;
                break;
            }
        }

        if (!inCircle && parent.find(next) == parent.end()) {
            parent[next] = current;
            q.push(next);
        }
    }
}
```

Listing 3: BFS algoritam za pronalaženje slobodne putanje

3.2 Rekonstrukcija i crtanje

Nakon što BFS stigne do ciljne tačke (`end`), putanja se rekonstruiše unazad koristeći mapu `parent`, koja za svaku tačku čuva koordinate njenog prethodnika.

- **Skaliranje:** Koordinate iz logičkog prostora algoritma skaliraju se faktorom `skala` kako bi popunile prozor aplikacije.
- **Iscrtavanje:** Putanja se crta kao serija povezanih linija (`DrawLineEx`) zelene boje, dok se krugovi iscrtavaju sa polu-providnim efektom (`Fade`) kako bi se jasno videlo preklapanje i ivice barijera.

```
if (ok && !pathPoints.empty()) {
    for (size_t i = 0; i < pathPoints.size() - 1; ++i) {
        DrawLineEx(pathPoints[i], pathPoints[i+1], 4.0f, GREEN);
    }
}
```

Listing 4: Iscrtavanje putanje u Raylib-u

4 Vizuelni prikaz i analiza test primera

U nastavku su prikazani karakteristični scenariji testiranja algoritma. Svaki primer ilustruje specifičan aspekt detekcije prohodnosti i iscrtavanja putanje.

- **Primer 1: Uspešna prohodnost**

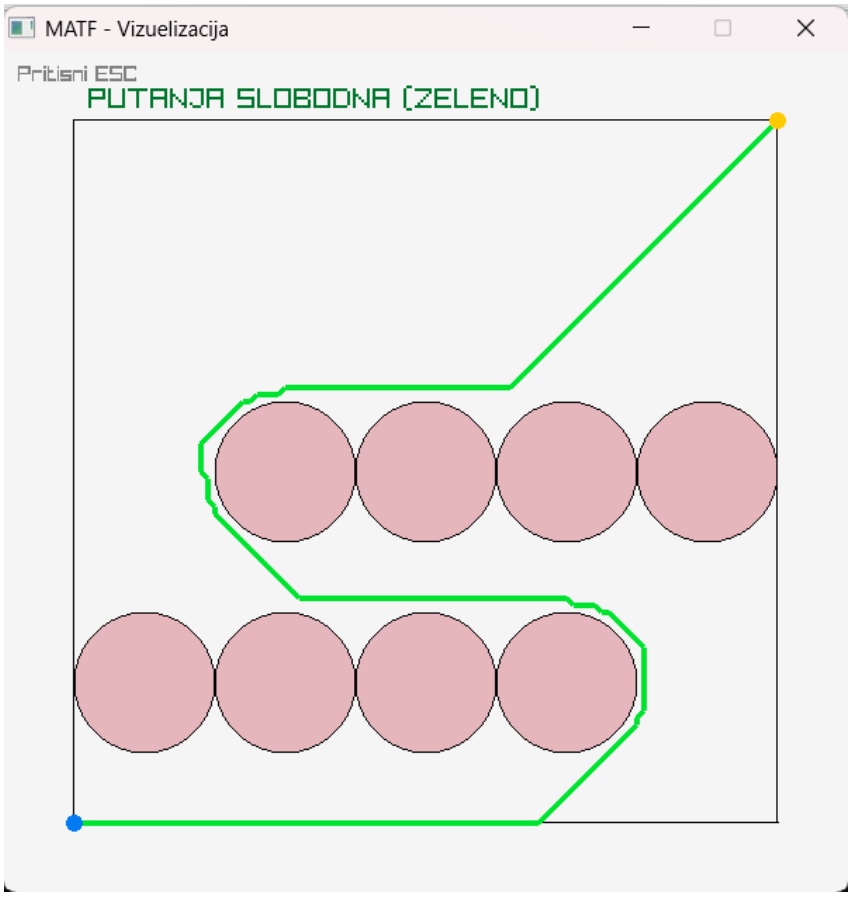

Figure 1: Uspešno pronađena putanja kroz kompleksan raspored prepreka

Na slici 1 vidimo kompleksniji raspored krugova koji formiraju "S" kanal. Iako su krugovi gusto raspoređeni, algoritam ispravno detektuje da ne postoji neprekidan lanac koji spaja gornju-levu sa donjom-desnom ivicom. Grafički BFS uspešno pronalazi krivudavu putanju koja održava minimalnu distancu od prepreka zahvaljujući uvedenoj margini od 0.1.

- **Primer 2: Blokada preklapanjem**

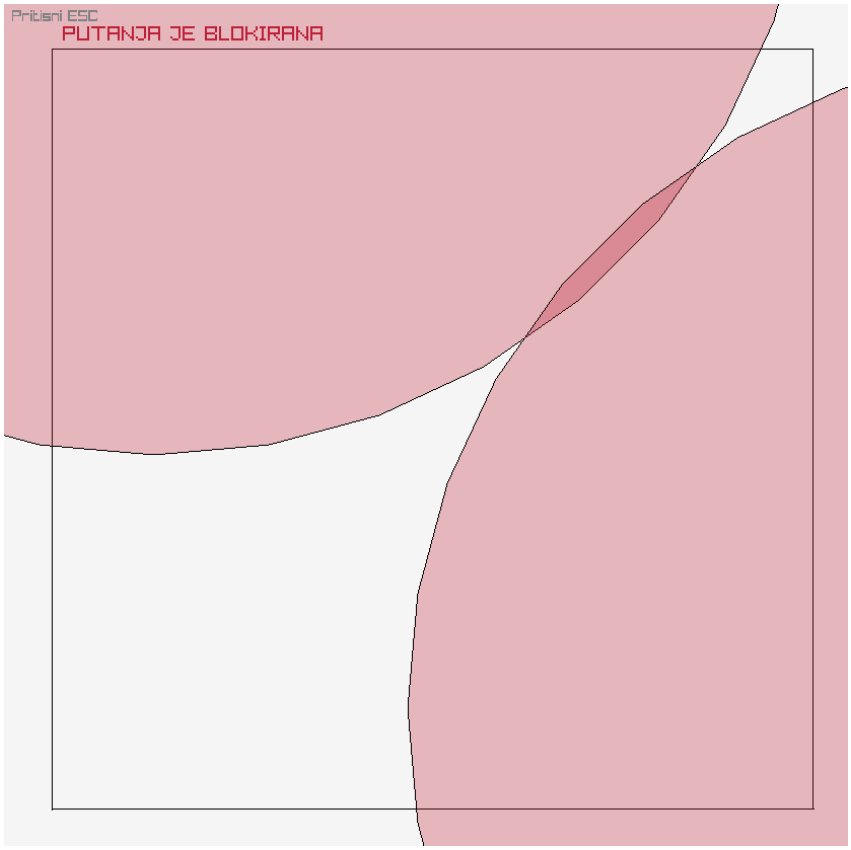

Figure 2: Barijera formirana preklapanjem velikih krugova

Slika 2 prikazuje scenario gde dva velika kruga seku unutrašnjost pravougaonika tako da se preklapaju u centralnom delu. Algoritam kroz `intersectCircles` i `intersectInside` funkcije prepoznaje da ovi krugovi čine nepremostivu barijeru između levog i desnog dela domena, čime se dokazuje da putanja od $(0, 0)$ do (X, Y) matematički ne može postojati.

- **Primer 3: Blokada dodirom**

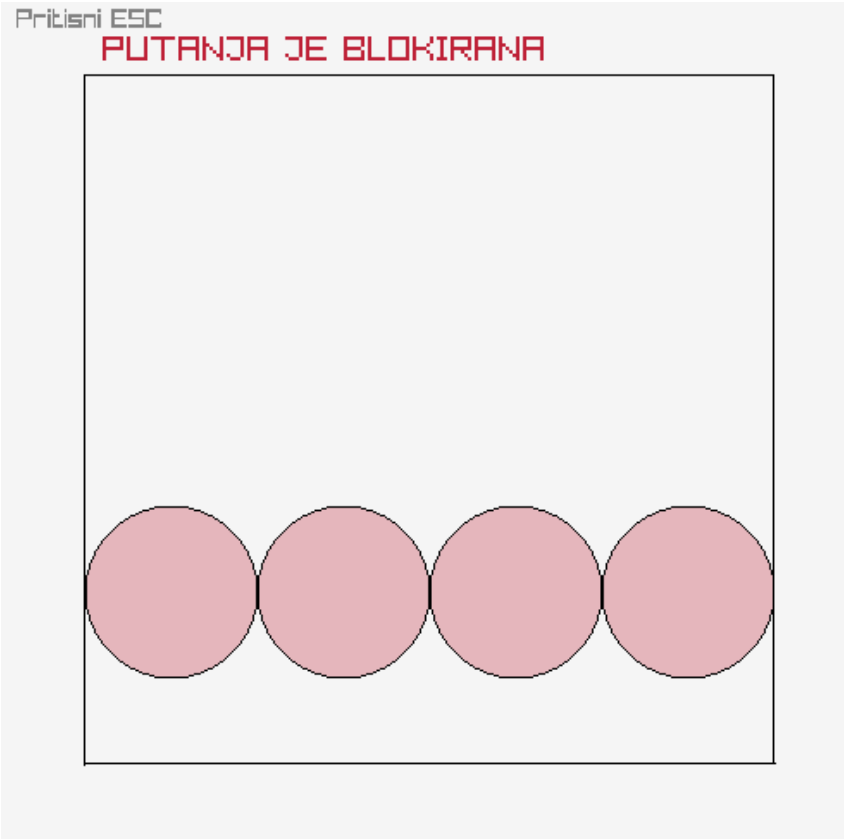

Figure 3: Horizontalna blokada formirana lancem krugova koji se dodiruju

Na slici 3 prikazan je niz manjih krugova koji se dodiruju u horizontalnom lancu. Ovaj primer je bio ključan za testiranje robusnosti algoritma. Zahvaljujući preciznoj proveri sudara, sistem detektuje da lanac dodiruje levu i desnu ivicu pravougaonika, čime efikasno "preseca" prostor na dva nepovezana dela, bez obzira na to što su tačke dodira krugova infinitesimale.

5 Mogućnosti primene

Algoritam razvijen u ovom projektu, iako primarno fokusiran na geometrijsku prohodnost unutar pravougaonog domena, ima širok spektar primene u realnim inženjerskim i istraživačkim problemima. Neke od ključnih oblasti uključuju:

- **Autonomni sistemi i robotika:** Planiranje putanje (*Path Planning*) je fundamentalni problem u robotici. Ovaj algoritam se može koristiti za proveru prohodnosti mobilnog robota u skladištima ili fabrikama gde su prepreke (stubovi, oprema, radnici) aproksimirane krugovima. Grafička vizuelizacija puta omogućava agentu da pronađe sigurnu distancu od prepreka, čime se minimizuje rizik od kolizije.
- **Razvoj video igara:** Iako moderni naslovi koriste kompleksne navigacione mreže (*NavMesh*), ovaj pristup je idealan za jednostavnije 2D sisteme, strateške igre ili proceduralno generisane nivoe. Omogućava brzo testiranje prohodnosti nivoa pre nego što se on uopšte prikaže igraču, štedeći procesorske resurse.
- **Projektovanje štampanih ploča (PCB Design):** U elektronici, krugovi mogu predstavljati zone zabranjenog rutiranja (*keep-out zones*) zbog toplote ili elektromagnetnih smetnji. Algoritam može automatski proveriti da li je moguće provući provodnik između dve tačke a da se ne naruši integritet signala ulaskom u zabranjenu zonu.
- **Sistemi za nadzor i pokrivenost:** Inverzno posmatrano, ako krugovi predstavljaju domet senzora ili sigurnosnih kamera, algoritam može otkriti "slepe tačke" — putanje kojima se uljez može kretati kroz štićeni prostor a da ostane neprimećen (tzv. *Maximum Breach Path*).

References

[1] LeetCode Problem 3235: *Check if the Rectangle Corner Is Reachable.*
Dostupno na: https://leetcode.com/problems/check-if-the-rectangle-corner-is-reachable/description/

[2] Raylib - *A simple and easy-to-use library to enjoy videogames programming.*
Dostupno na: https://www.raylib.com/